

Platform for Complete Machine Learning Lifecycle

Part 1

Course Overview

This course covers two of the most popular open-source platforms for MLOps (Machine Learning Operations): ML flow and Hugging Face. We'll go through the foundations on what it takes to get started in these platforms with basic model and dataset operations. You will start with ML flow using projects and models with its powerful tracking system and you will learn how to interact with these registered models from ML flow with full lifecycle examples. Then, you will explore Hugging Face repositories so that you can store datasets, models, and create live interactive demos.

Outline – Introduction to MLflow: How to Use MLflow Tracking - Part 1

- Overview of ML development challenges
- Concepts and Motivations
- How MLflow tackles these
- MLFlow Components
 - MLflow Tracking
 - How to use MLflow Tracking APIs
 - Use Databricks Community Edition
 - Explore MLflow UI
 - Tutorials
- Q & A

Machine Learning Development is Complex

Traditional Software vs. Machine Learning

Traditional Software

- Goal: Meet a functional specification
- Quality depends only on code
- Typically pick one software stack w/ fewer libraries and tools
- Limited deployment environments

Machine Learning

- Goal: Optimize metric(e.g., accuracy. Constantly experiment to improve it
- Quality depends on input data and tuning parameters
- Compare + combine many libraries, model
- Diverse deployment environments

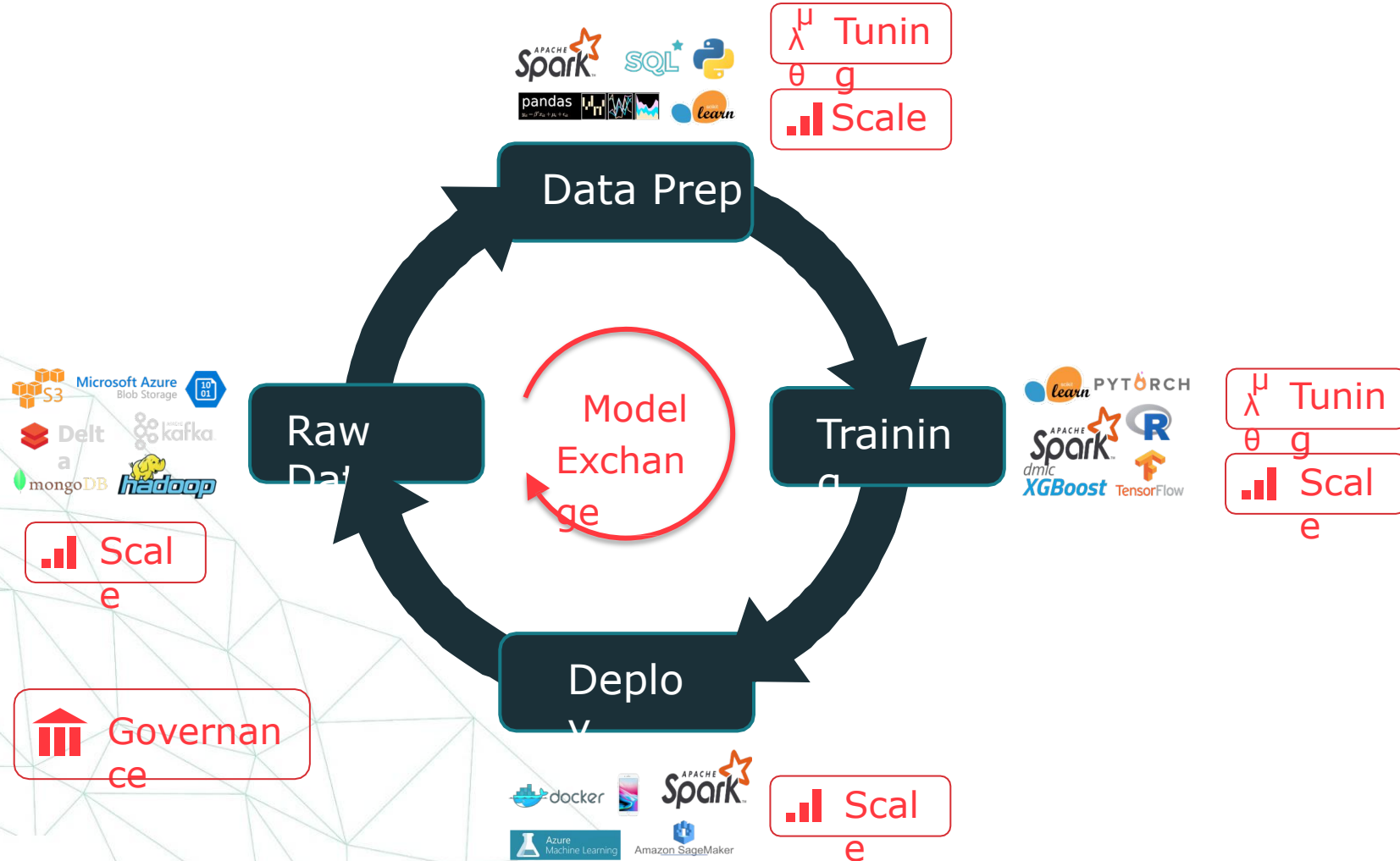
Custom Machine Learning Platforms

Some Big Data Companies

- + Standardize the data prep / training / deploy loop: if you work with the platform, you get these!
 - Limited to a few algorithms or frameworks
 - Tied to one company's infrastructure
 - Out of luck if you left the company....

Can we provide similar benefits in an **open** manner?

Machine Learning Lifecycle



Machine learning operations (MLOps)

- ML Engineering culture and practice that aims at unifying ML System development (Dev) and ML system operations (Ops)
- Tools and principles to support workflow standardization and automation through the ML system lifecycle (e.g. with pipelines)

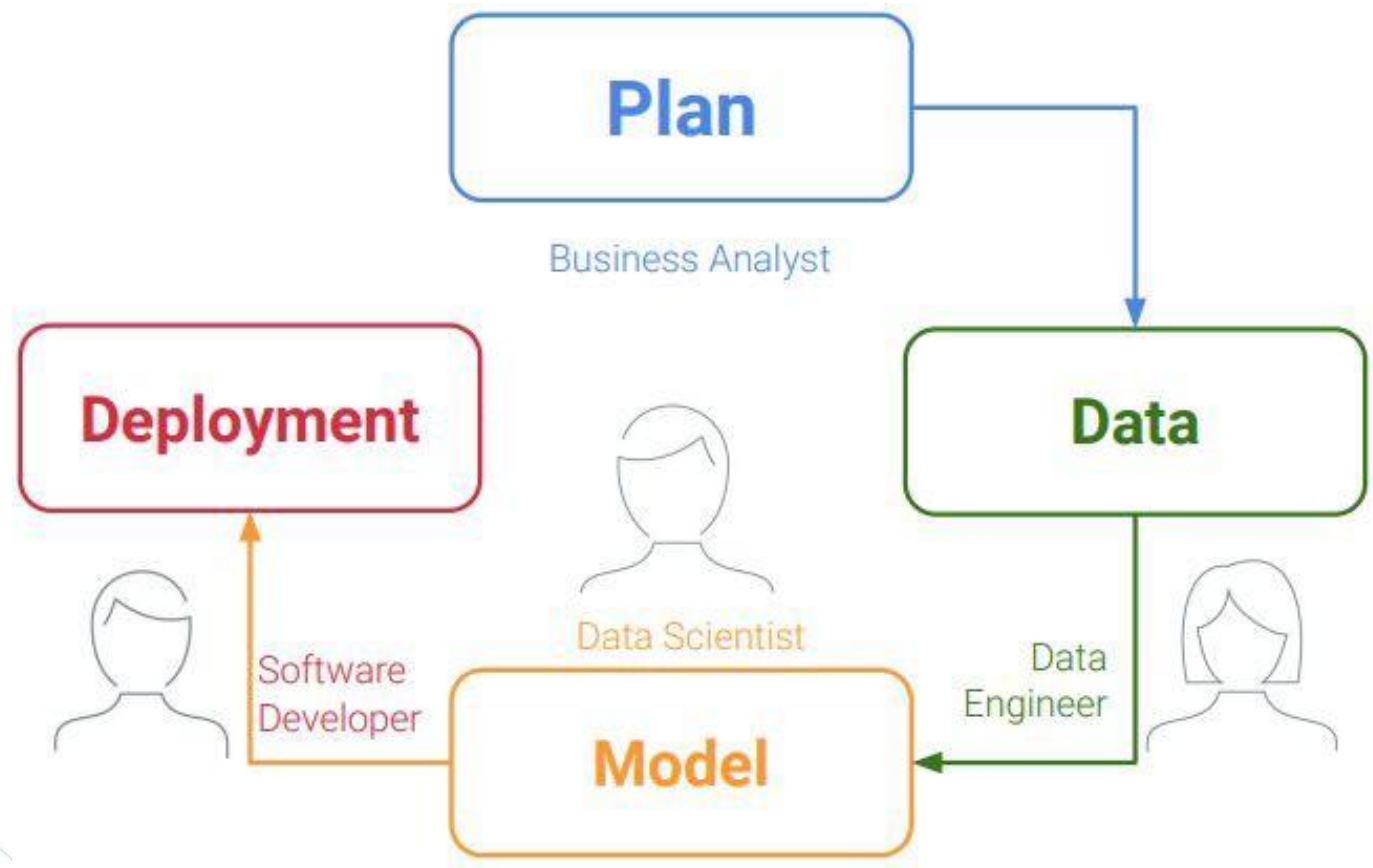


Machine learning lifecycle

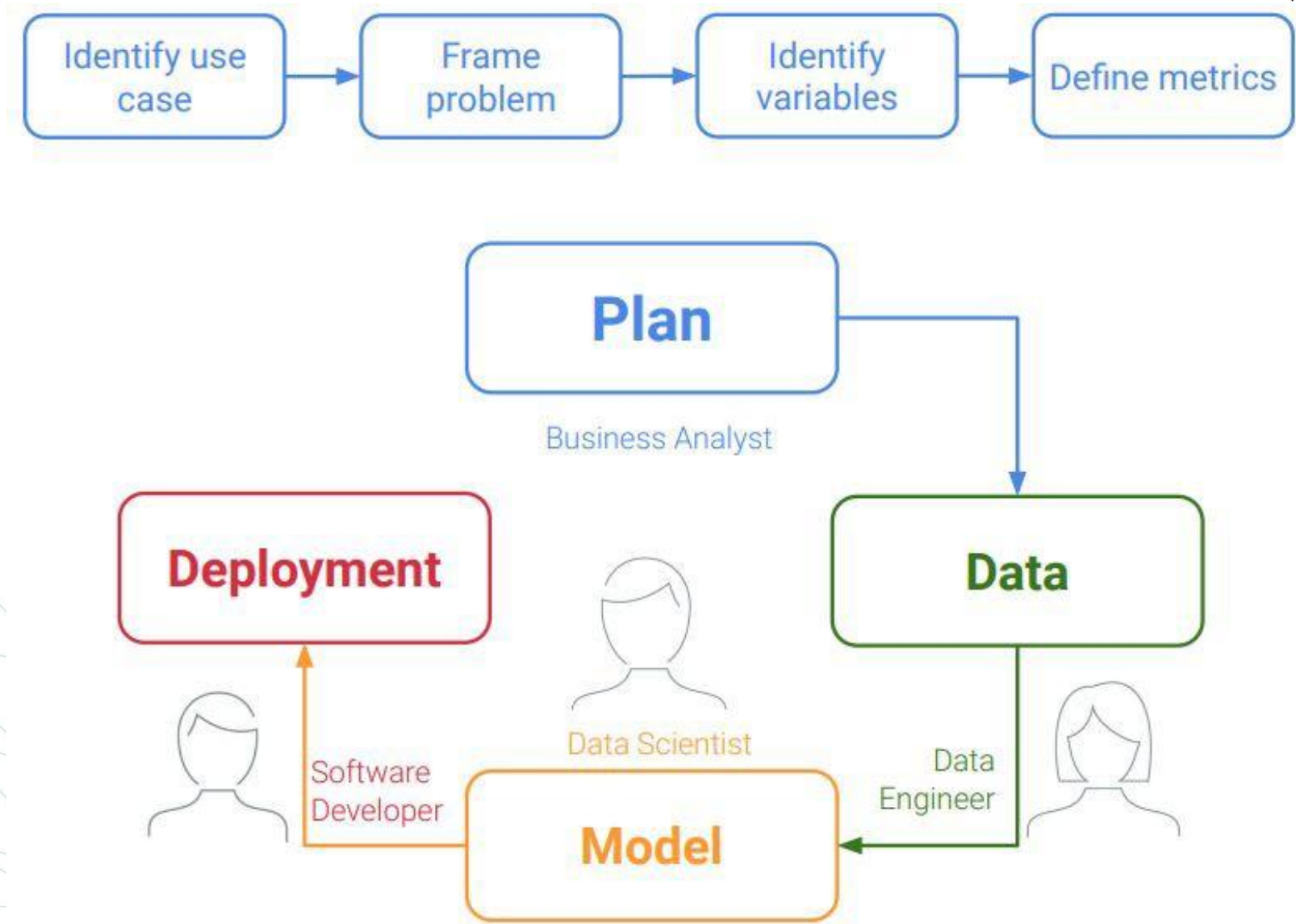
Lifecycle

of an ML System

Plan | Data | Model |
Deployment

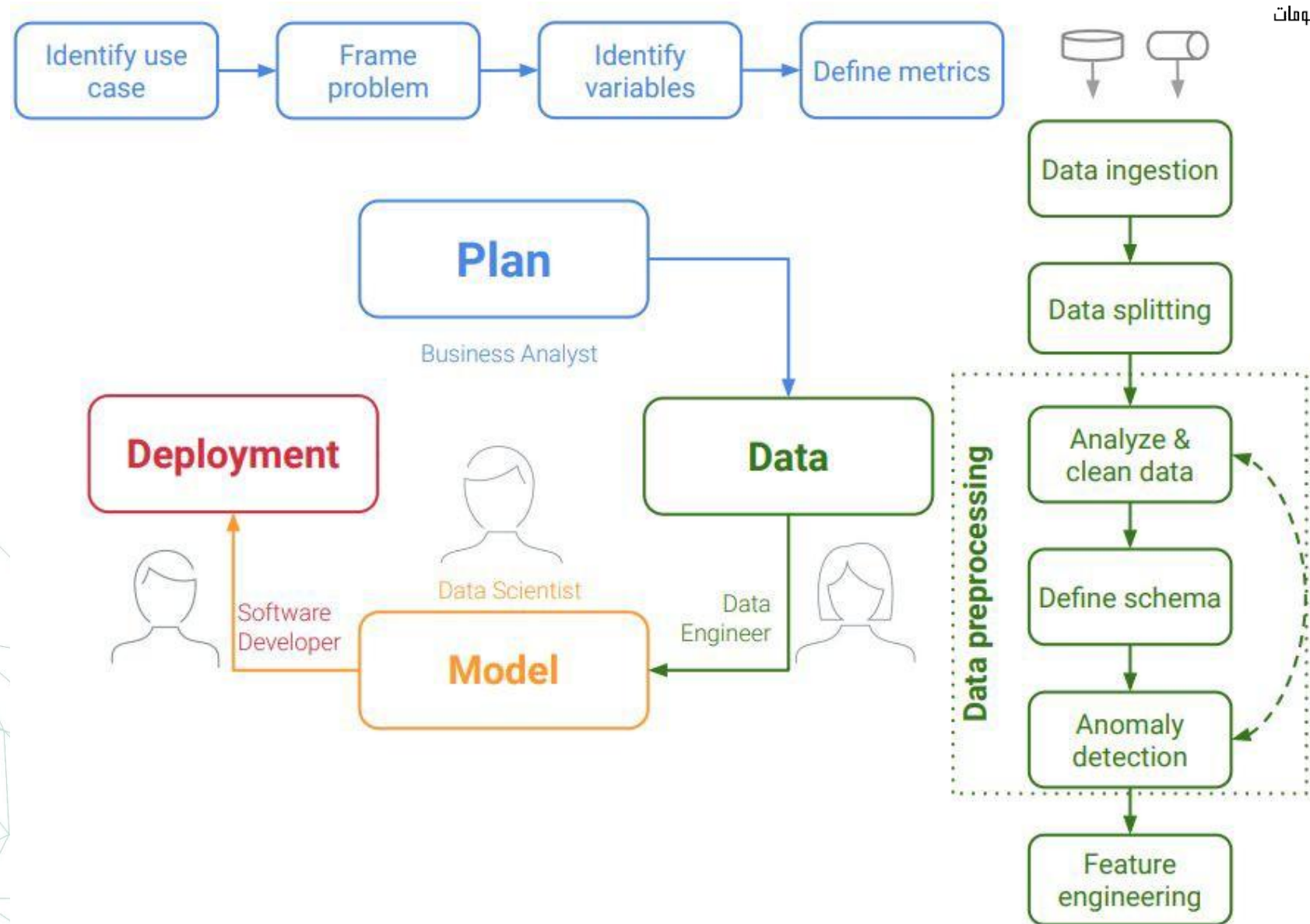


Lifecycle of an ML System Plan | Data | Model | Deployment



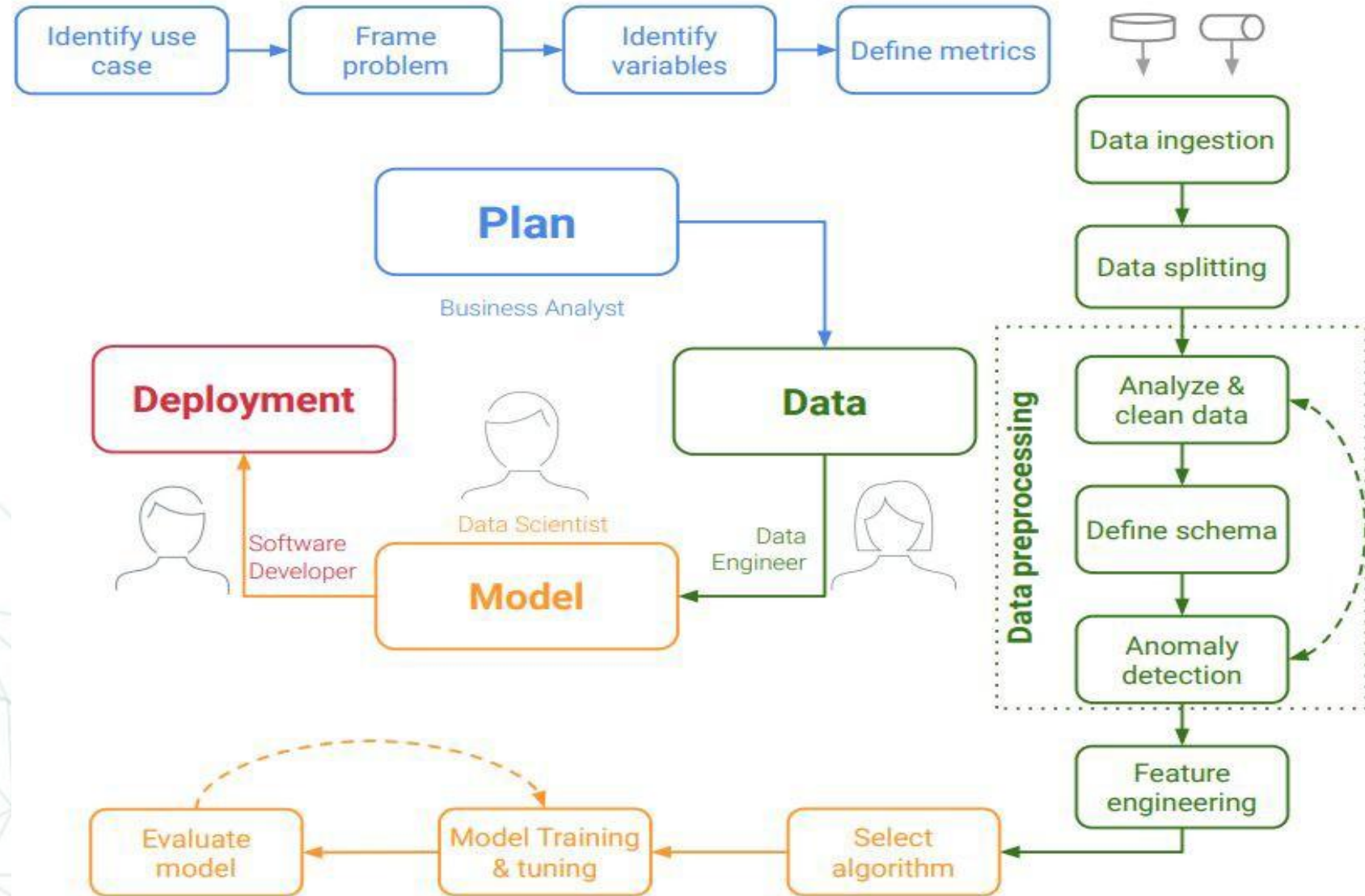
Lifecycle of an ML System

Plan | Data | Model
| Deployment



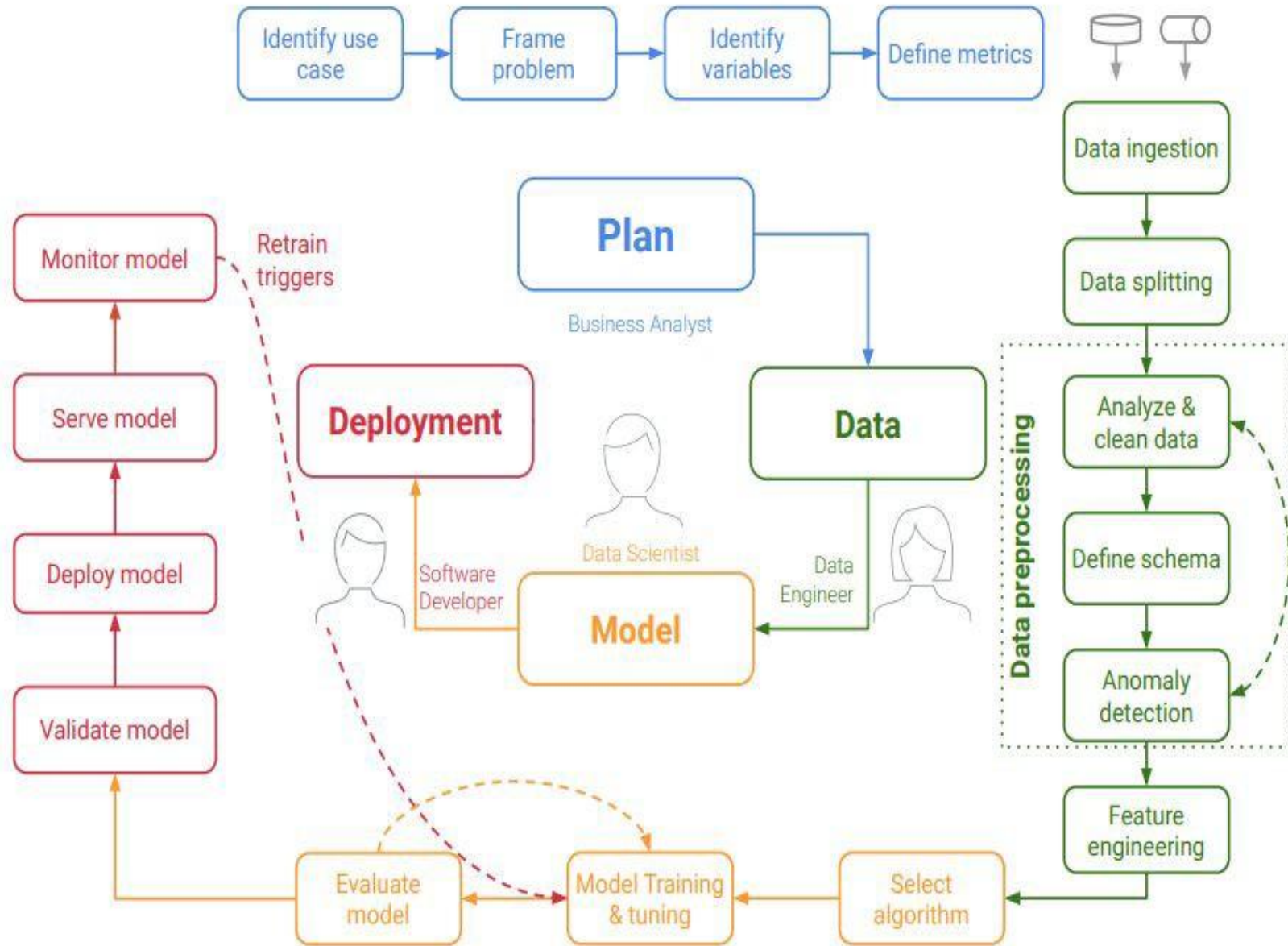
Lifecycle of an ML System

Plan | Data | Model
| Deployment



Lifecycle of an ML System

Plan | Data | Model | Deployment

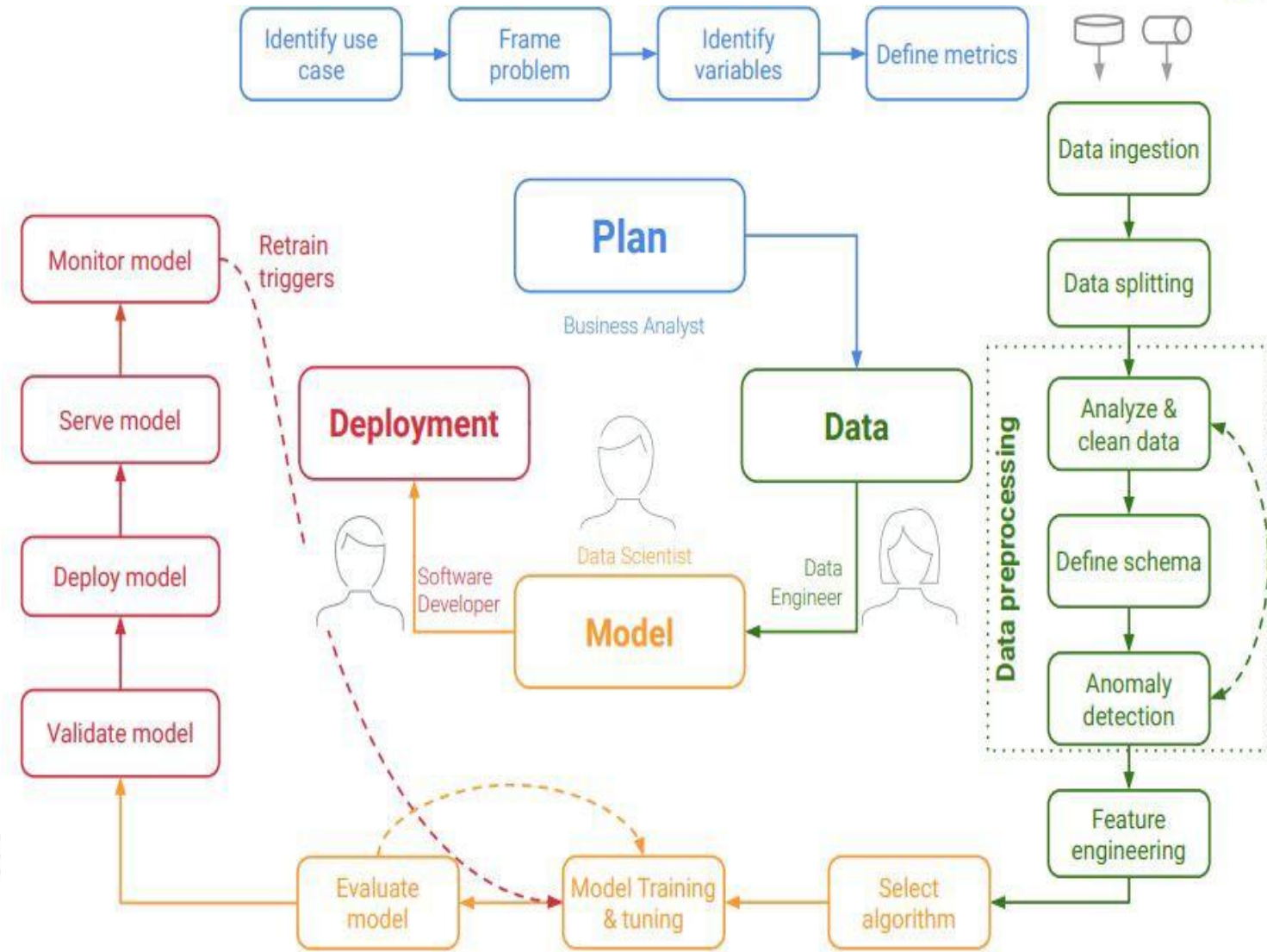


Lifecycle

of an ML System Plan |
Data | Model |
Deployment

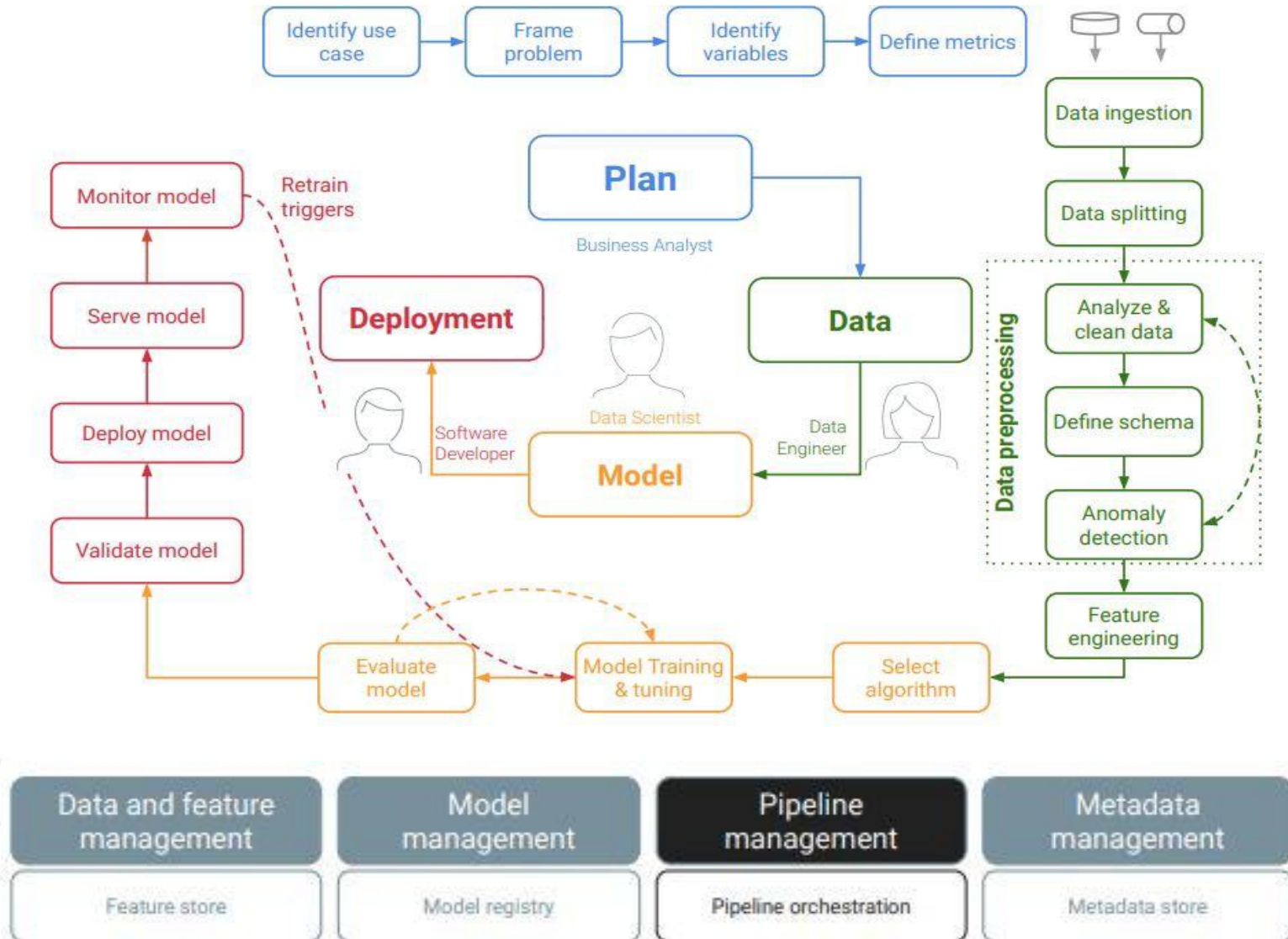
Common issues which lead to a PoC to production gap

- Lack of reuse and duplication
- Inconsistency (data, code, models)
- Manual and slow transition from PoC to production



Lifecycle of an ML System Plan | Data | Model | Deployment

MLOps components



Introducing

mlflow

Open machine learning platform

Works with popular ML library & language

Runs the same way anywhere (e.g., any cloud or locally)

Designed to be useful for 1 or 1000+ person orgs

Simple. Modular. Easy-to-use.

Offers positive developer experience to get started!

MLflow Design Philosophy

API-First

- Submit runs, log models, metrics, etc. from popular library & language
- Abstract “model” lambda function that MLflow can then deploy in many places (Docker, Azure ML, Spark UDF)
- Open interface allows easy integration from the community

**Key enabler: built around
Programmatic APIs, REST APIs & CLI**

Modular Design

- Allow different components individually (e.g., use MLflow’s project format but not its deployment tools)
- Not monolithic
- But Distinctive and Selective

**Key enabler: distinct components
(Tracking/Projects/Models/Registry)**

MLflow Components

mlflow Tracking

Record and query experiments: code, data, config, and results

databricks.com/mlflow

mlflow Projects

Package data science code in a format that enables reproducible runs on many platform



mlflow.org

mlflow Models

Deploy machine learning models in diverse serving environments



github.com/mlflow

new

mlflow Model Registry

Store, annotate and manage models in a central

twitter.com/mlflow



Key Concepts in MLflow Tracking

Parameters: key-value inputs to your code

Metrics: numeric values (can update over time)

Tags and Notes: information about a run

Artifacts: files, data, and models

Source: what code ran?

Version: what of the code?

Run: an instance of code that runs by MLflow

Experiment: {Run, ... Run}

Model Development without MLflow

```
data = load_text(file)
ngrams = extract_ngrams(data, N=n)
model = train_model(ngrams,
                    learning_rate=lr)
score = compute_accuracy(model)

print("For n=%d, lr=%f: accuracy=%f"
      % (n, lr, score))

pickle.dump(model, open("model.pkl"))
```

```
For n=2, lr=0.1: accuracy=0.71
For n=2, lr=0.2: accuracy=0.79
For n=2, lr=0.5: accuracy=0.83
For n=2, lr=0.9: accuracy=0.79
For n=3, lr=0.1: accuracy=0.83
For n=3, lr=0.2: accuracy=0.82
For n=4, lr=0.5: accuracy=0.75
...
```

What if I
expand the
input data?

Model Development without MLflow

```
data = load_text(file)
ngrams = extract_ngrams(data, N=n)
model = train_model(ngrams,
                    learning_rate=lr)
score = compute_accuracy(model)

print("For n=%d, lr=%f: accuracy=%f"
      % (n, lr, score))

pickle.dump(model, open("model.pkl"))
```

```
For n=2, lr=0.1: accuracy=0.71
For n=2, lr=0.2: accuracy=0.79
For n=2, lr=0.5: accuracy=0.83
For n=2, lr=0.9: accuracy=0.79
For n=3, lr=0.1: accuracy=0.83
For n=3, lr=0.2: accuracy=0.82
For n=4, lr=0.5: accuracy=0.75
...
```

What if I tune
this other
parameter?

Model Development without MLflow

```
data = load_text(file)
ngrams = extract_ngrams(data, N=n)
model = train_model(ngrams,
                    learning_rate=lr)
score = compute_accuracy(model)

print("For n=%d, lr=%f: accuracy=%f"
      % (n, lr, score))

pickle.dump(model, open("model.pkl"))
```

```
For n=2, lr=0.1: accuracy=0.71
For n=2, lr=0.2: accuracy=0.79
For n=2, lr=0.5: accuracy=0.83
For n=2, lr=0.9: accuracy=0.79
For n=3, lr=0.1: accuracy=0.83
For n=3, lr=0.2: accuracy=0.82
For n=4, lr=0.5: accuracy=0.75
...
```

What version
of my code
was this

result from

Model Development *with* MLflow is *Simple*!

```
import mlflow
data = load_text(file)
ngrams = extract_ngrams(data, N=n)
model = train_model(ngrams,
                    learning_rate=lr)
score = compute_accuracy(model)
with mlflow.start_run():
    mlflow.log_param("data_file", file)
    mlflow.log_param("n", n)
    mlflow.log_param("learn_rate", lr)
    mlflow.log_metric("score", score)
    mlflow.sklearn.log_model(model)
```

```
$ mlflow ui
```

mlflow

Experiments

Default

Default

Experiment ID: 0

Artifact Location: /home/katherine/mlruns/0

Search Runs: metrics.rmse < 1 and params.model = "tree"

Search

Filter Params: alpha, lr

Filter Metrics: rmse, r2

Clear

36 matching runs

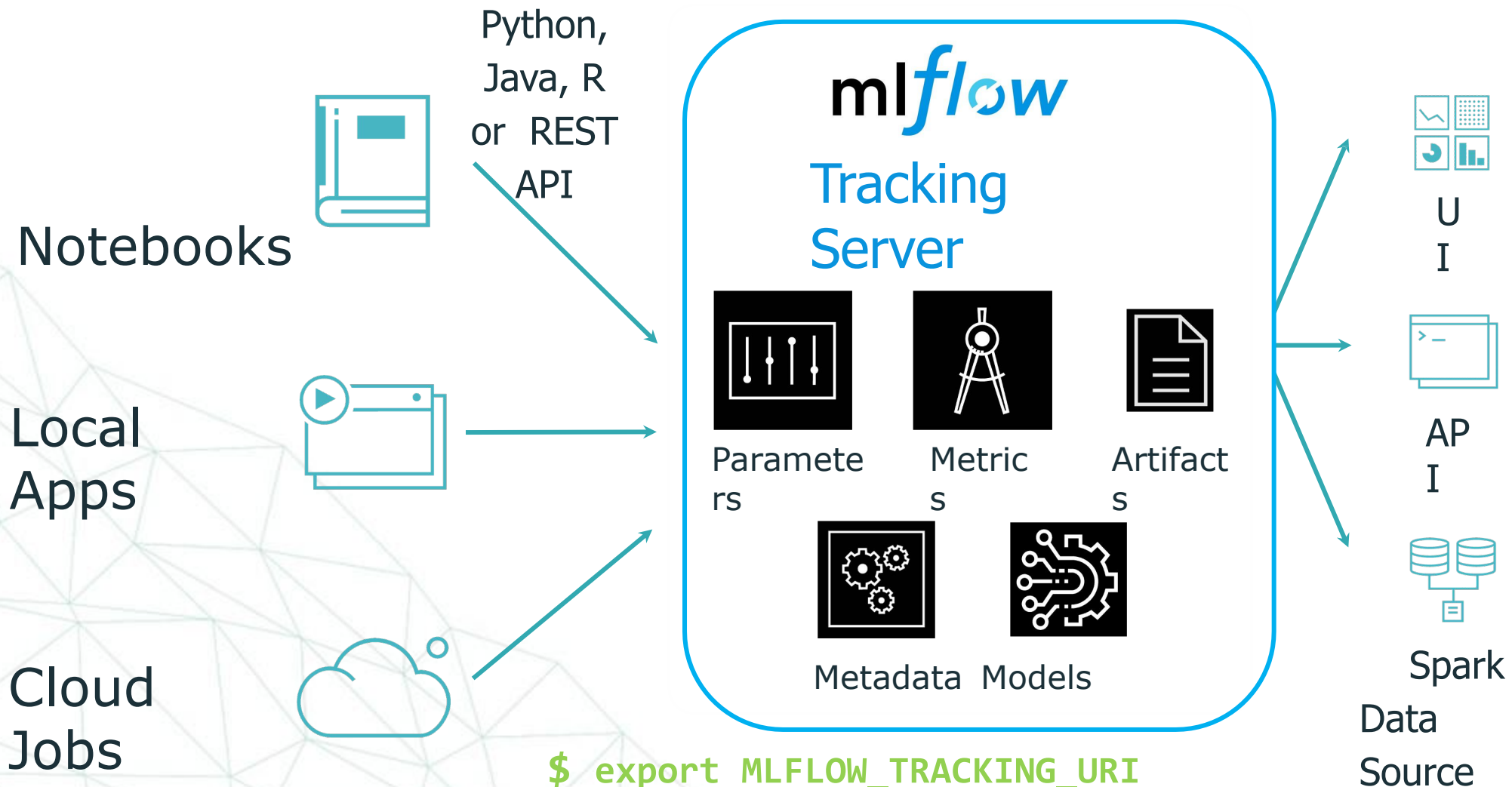
Complete Selected

Download CSV

	Date	User	Source	Version	Parameters		Metrics		
					alpha	l1_ratio	mae	r2	rmse
☐	2018-07-19 03:26:53	root	azure-demo1		0.01	0.55	0.596	0.25	0.762
☐	2018-07-19 03:26:39	root	azure-demo		0.01	0.55	0.596	0.25	0.762
☐	2018-07-19 03:26:14	root	azure-demo		0.01	0.55	0.596	0.25	0.762
☐	2018-07-19 03:25:51	root	azure-demo		0.01	0.75	0.597	0.25	0.762
☐	2018-07-19 03:25:42	root	azure-demo		0.01	0.04	0.591	0.256	0.759
☐	2018-07-18 02:09:54	root	azure-demo		0.01	1.0	0.597	0.249	0.762
☐	2018-07-18 02:09:29	root	azure-demo		0.01	0.75	0.597	0.25	0.762
☐	2018-07-18 02:08:52	root	azure-demo		0.01	0.01	0.591	0.257	0.759
☐	2018-07-17 08:13:37	root	azure-demo		0.01	0.01	0.591	0.257	0.759
☐	2018-07-17 08:13:34	root	azure-demo		0.01	1.0	0.597	0.249	0.762
☐	2018-07-17 08:13:30	root	azure-demo		0.01	0.75	0.597	0.25	0.762
☐	2018-07-17 08:13:27	root	azure-demo		0.01	0.01	0.591	0.257	0.759
☐	2018-07-17 08:08:05	root	azure-demo		0.01	0.01	0.591	0.257	0.759

Track parameters,
metrics, output files &
code version

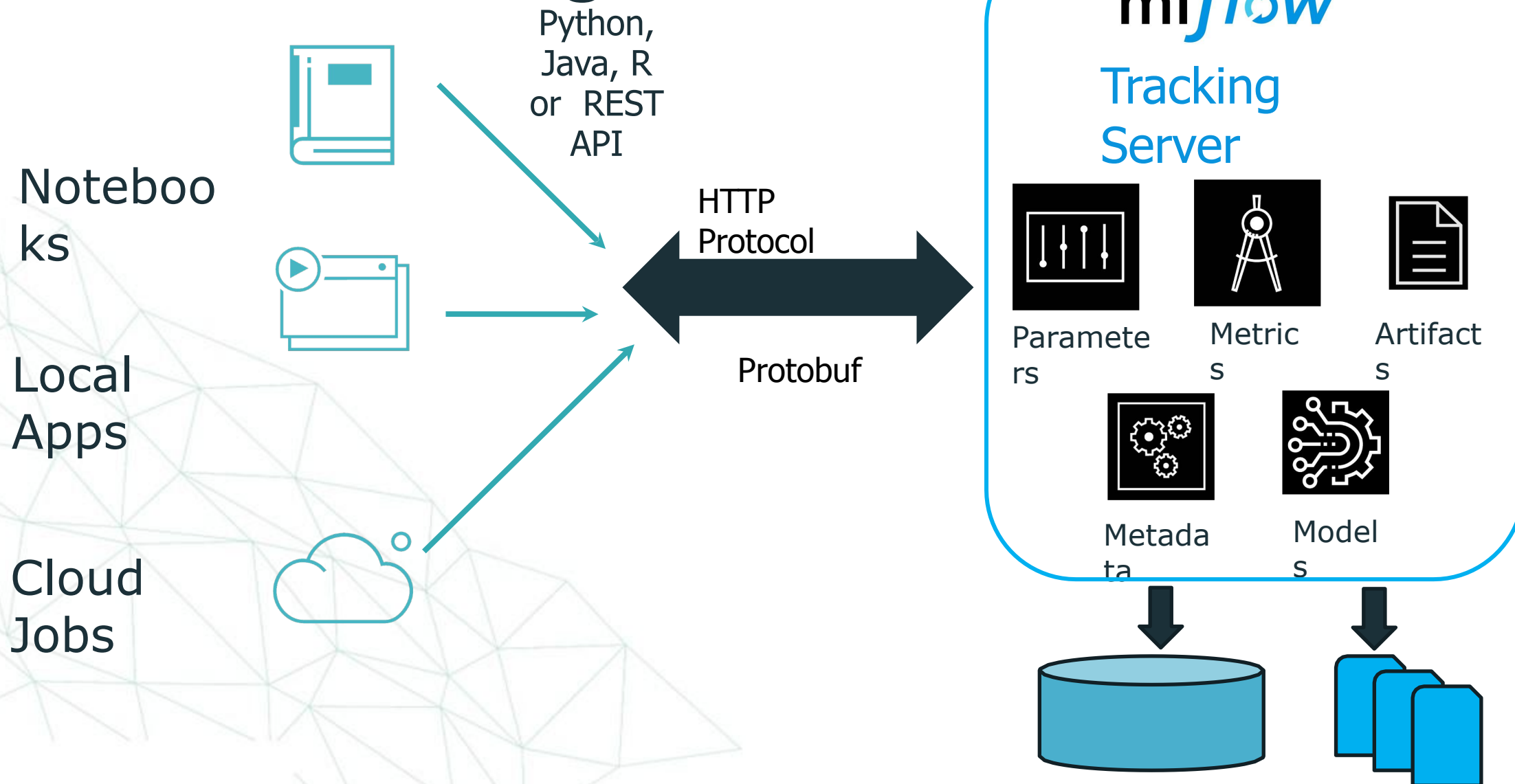
MLflow Tracking



```
$ export MLFLOW_TRACKING_URI  
<URI>
```

```
mlflow.set_tracking_uri(URI)
```

MLflow Tracking



MLflow Tracking Backend Stores

Entity (Metadata) Store

- FileStore (local filesystem)
 - **mlruns** directory by default
- SQLStore (via SQLAlchemy)
 - PostgreSQL, MySQL, SQLite
- MLflow Plugins Scheme
 - Customized Entity Metastore
- Managed MLflow on Databricks
 - MySQL on AWS and Azure

Artifact Store

- Local Filesystem
 - **mlruns** directory
- S3 backed store
- Azure Blob storage
- Google Cloud Storage
- DBFS artifact repo

MLFlow set of APIs

Fluent MLflow APIs

MLflowClient

MLflow REST API

Python

- High-level operations for runs and experiments
- Model Flavor APIs

Java

- MLflowContext
- Experiments, runs, search, etc

R

- Experiments, runs, search etc



- Low Level CRUD interface to experiments and runs
- `import mflow.tracking`
- `client = MLflowClient(**kwargs)`
- Metric, Param etc



- Make REST class to Tracking server with endpoints
- `https://<tracking_server>/api/..`
- `/2.0/mlflow/experiments/create`
- `/2.0/mlflow/experiments/get`
- `/2.0/mlflow/experiments/get-by-name`

What Did We Talk About?

- Modular Components greatly simplify the ML lifecycle
- Easy to install and use & Great Developer Experience
- Develop & Deploy locally and track locally or remotely
- Available APIs: Python, Java & R (Soon Scala)
- Visualize experiments and compare runs
- Centrally register and manage model lifecycle

mlflowTM

1,363 commits

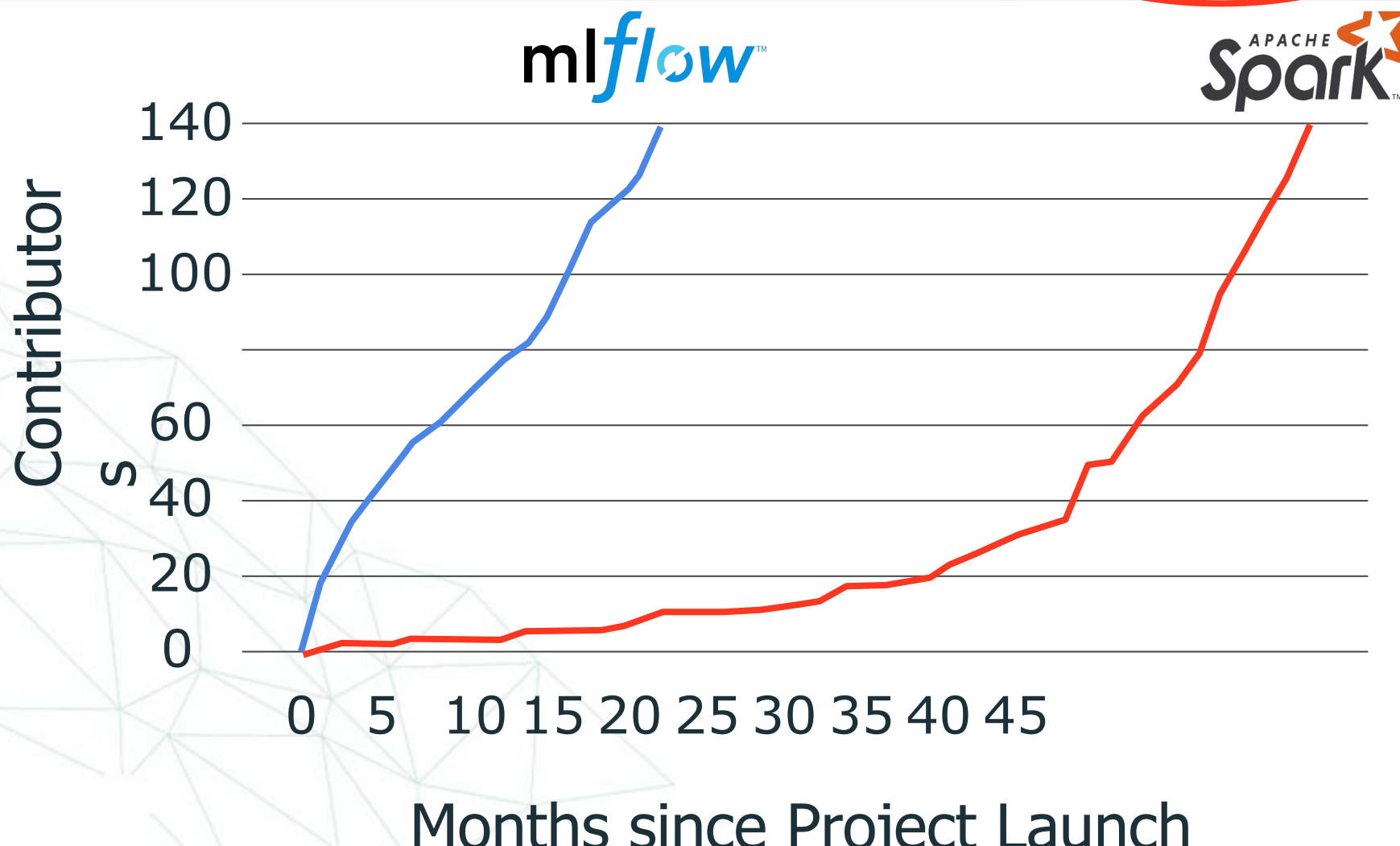
37 branches

0 packages

28 releases

200 contributors

Apache-2.0



Learning More About MLflow & Get Involved!

- `pip install mlflow`—to get started
- Find docs & examples at mlflow.org
- Peruse code and contribute at [MLflow Github](https://github.com/mlflow/mlflow)
- Join the [Slack channel](#)
- More [MLflow tutorials](#)

Thank you!



mlflow

Platform for Complete Machine Learning Lifecycle

Part 2

Outline – Introduction to MLflow: MLflow Projects & Models - Part 2

- MLFlow Component
 - MLflow Projects & Models
 - Concepts and Motivations
 - MLflow on Databricks Community Edition (DCE)
 - Explore MLflow UI
 - Tutorials
- Q & A

mlflow Tracking

Record and query experiments: code, data, config, and results

databricks.com/mlflow

mlflow Projects

Package data science code in a format that enables reproducible runs on any platform



mlflow.org

mlflow Models

Deploy machine learning models in diverse serving environments



github.com/mlflow/mlflow

new

mlflow Model Registry

Store, annotate and manage models in a central repository



twitter.com/mlflow

Diverse set of tools

PYTORCH

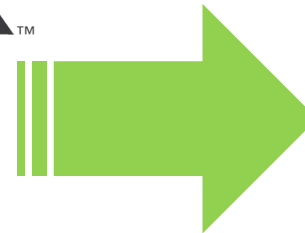
dmlc
XGBoost

TensorFlow

scikit
learn

APACHE
Spark™

R



mlflow™
Projects

Package data science
code in a format that
enables reproducible
runs on any platform

Diverse set of environments



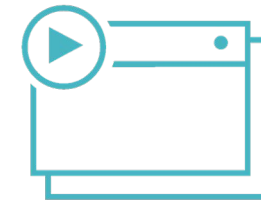
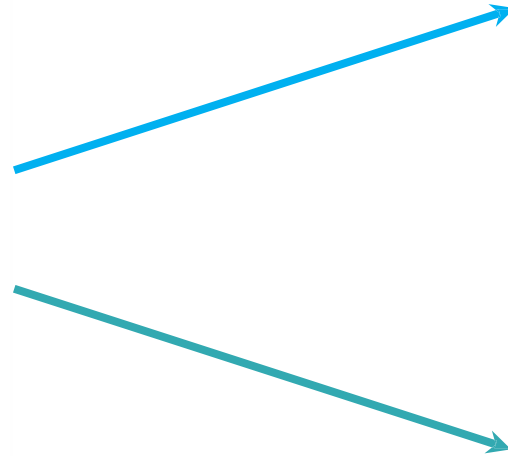
docker

Azure
Machine Learning



databricks

Challenge: ML results difficult to
reproduce



Local
Execution



Remote
Execution



databricks



1. Example MLflow Project File

my_project
└─ MLproject

```
conda_env: conda.yaml

entry_points:
  main:
    parameters:
      training_data: path
      lambda: {type: float, default: 0.1}
    command: python main.py {training_data} {lambda}
```

└─ conda.yaml
└─ main.py
└─ model.py

...

```
$ mlflow run git://<my_project>.git -P lambda=0.2
```

```
mlflow.run("git://<my_project>", parameters={..})
```

```
mlflow run . -e main -P lambda=0.2
```

2. Example Conda.yaml

my_project/

├── MLproject

├── conda.yaml

├── main.py

└── model.py

....

```
channels:
- defaults
dependencies:
- python=3.7.3
- scikit-learn=0.20.3
- pip:
  - Mlflow
  - cloudpickle==0.8.
name: mlflow-env
```

MLflow Projects

Packaging format for reproducible ML runs

Any code folder or GitHub repository

- MLproject file with project configuration

Defines dependencies for reproducibility

- Conda (+ R, Docker, ...) dependencies can be specified in MLproject
- Reproducible in (almost) any environment

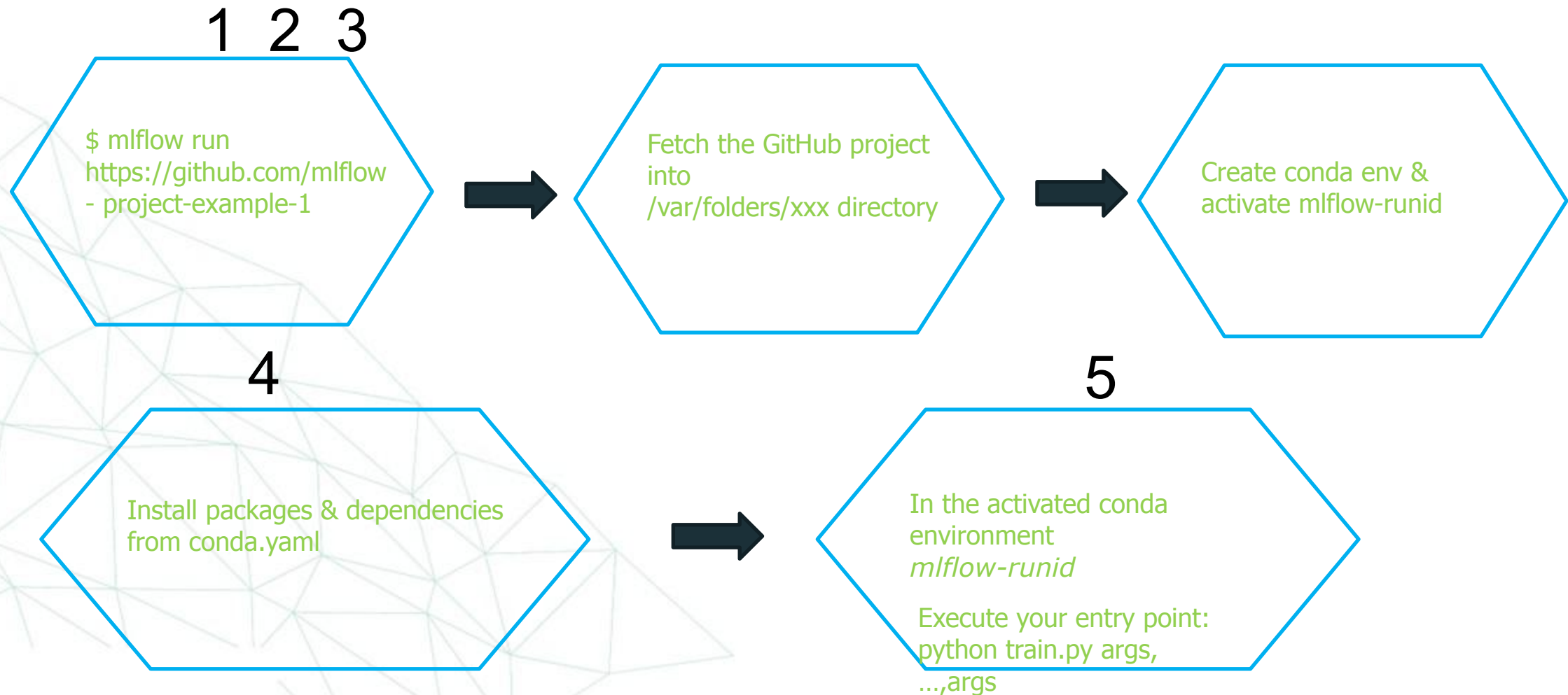
Execution API for running projects

- CLI / Python / R / Java
- Supports local and remote execution

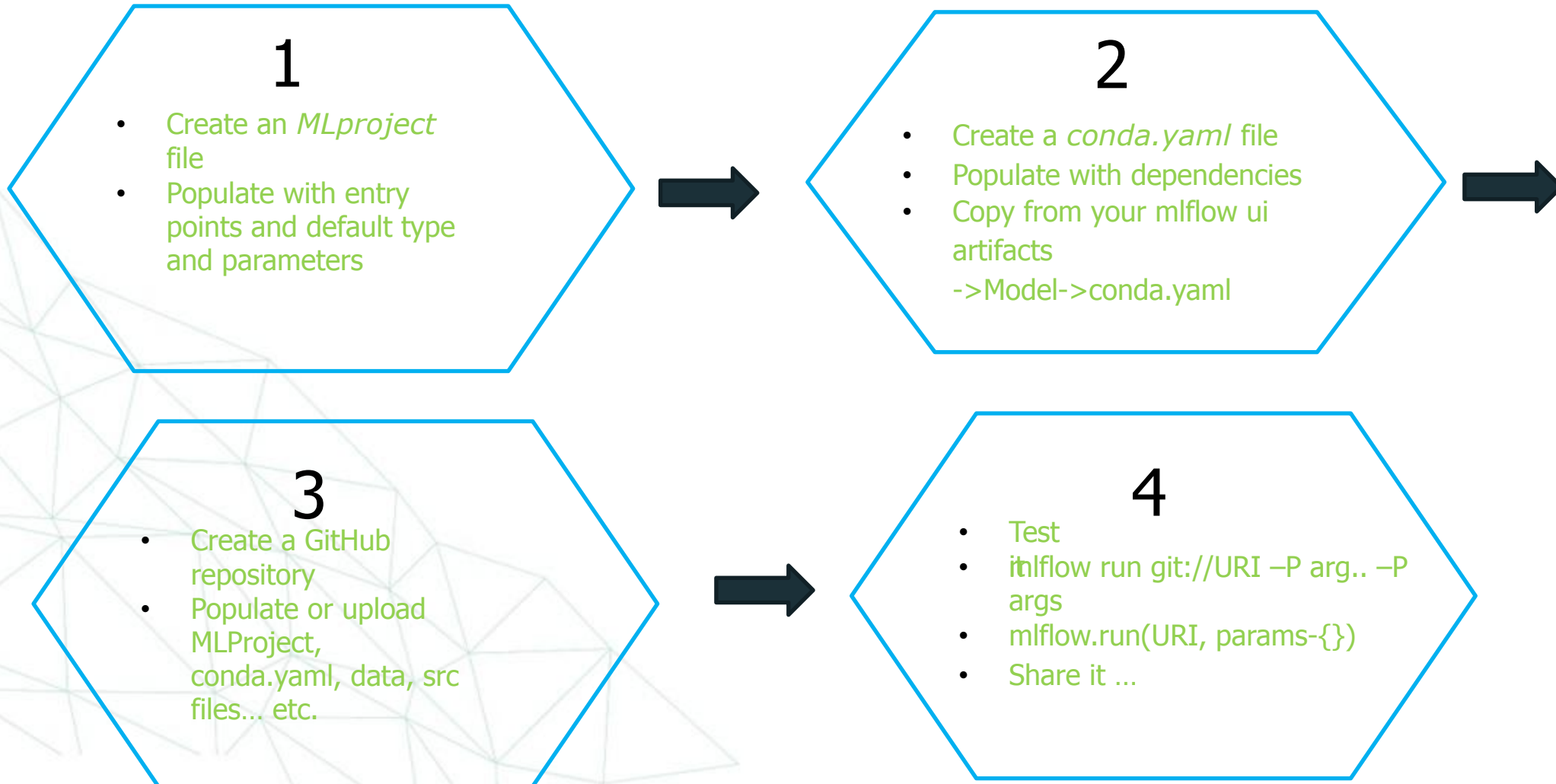
directory paths to
MLproject file

- `mlflow run -help` (CLI)
- `mlflow run https://github.com/dmatrix/jsd-mlflow-examples.git#keras/imdbclassifier` (CLI)
- `mlflow.run (<project_uri>, parameters={})` or `mlflow.projects.run((<project_uri>, parameters={})` (API)

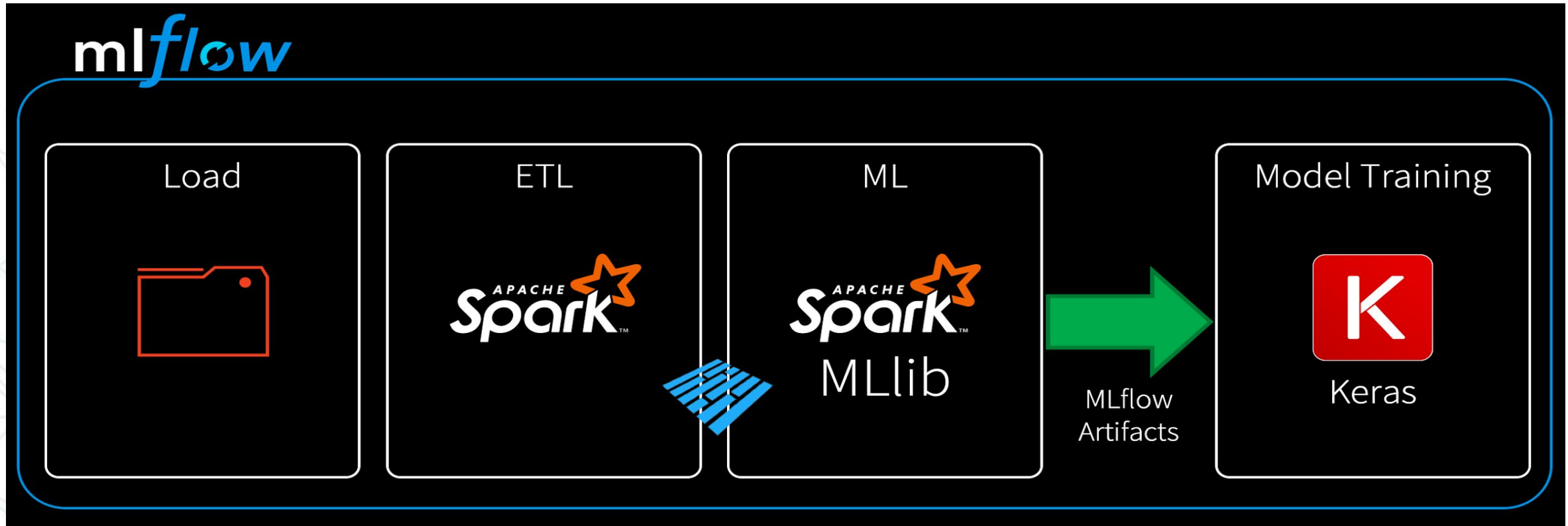
Anatomy of Mlflow Project Execution



How to build an Mlflow Project



MLflow Project: Create Multi-Step Workflow



https://github.com/mlflow/mlflow/tree/master/examples/multistep_workflow



```
1 name: multistep_example
2
3 conda_env: conda.yaml
4
5 entry_points:
6   load_raw_data:
7     command: "python load_raw_data.py"
8
9   etl_data:
10    parameters:
11      ratings_csv: path
12      max_row_limit: {type: int, default: 100000}
13    command: "python etl_data.py --ratings-csv {ratings_csv} --max-row-limit {max_row_limit}"
14
15   als:
16    parameters:
17      ratings_data: path
18      max_iter: {type: int, default: 10}
19      reg_param: {type: float, default: 0.1}
20      rank: {type: int, default: 12}
21    command: "python als.py --ratings-data {ratings_data} --max-iter {max_iter} --reg-param {reg_param} --rank {rank}"
22
23   train_keras:
24    parameters:
25      ratings_data: path
26      als_model_uri: string
27      hidden_units: {type: int, default: 20}
28    command: "python train_keras.py --ratings-data {ratings_data} --als-model-uri {als_model_uri} --hidden-units {hidden_units}"
29
30   main:
31    parameters:
32      als_max_iter: {type: int, default: 10}
33      keras_hidden_units: {type: int, default: 20}
34      max_row_limit: {type: int, default: 100000}
35    command: "python main.py --als-max-iter {als_max_iter} --keras-hidden-units {keras_hidden_units}
36              --max-row-limit {max_row_limit}"
37
```

mlflow Tracking

Record and query experiments: code, data, config, and results

databricks.com/mlflow

mlflow Projects

Package data science code in a format that enables reproducible runs on any platform



mlflow.org

mlflow Models

Deploy machine learning models in diverse serving environments



github.com/mlflow/mlflow

new

mlflow Model Registry

Store, annotate and manage models in a central repository



twitter.com/mlflow



PYTORCH



dmlc
XGBoost

ML
Frameworks

N x M
Combination of
Model support
for all Serving
tools



Inference
Code

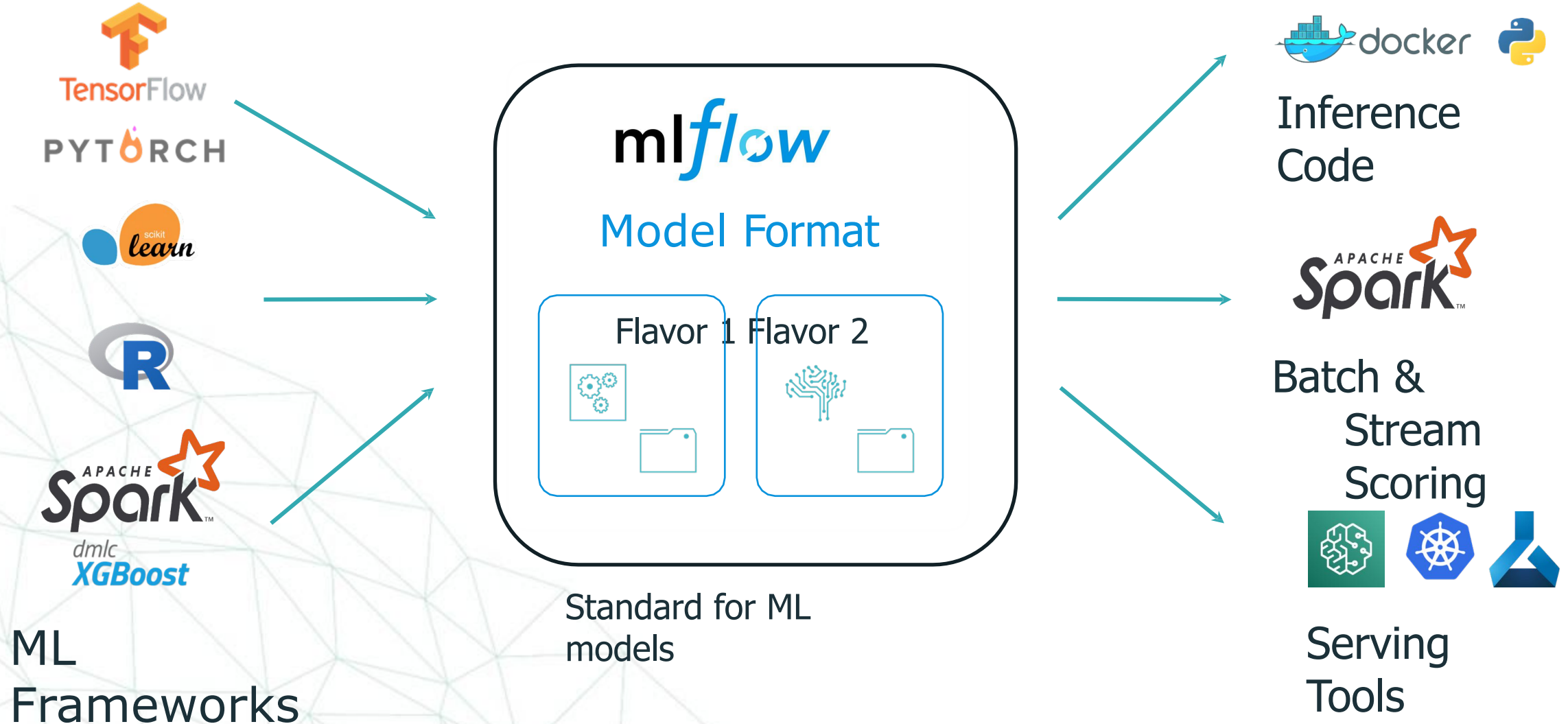


Batch & Stream
Scoring

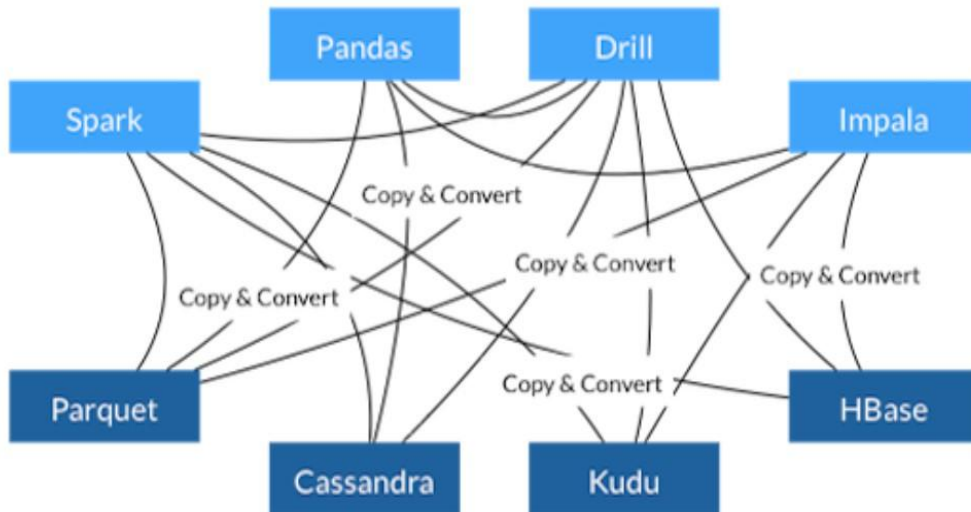


Serving
Tools

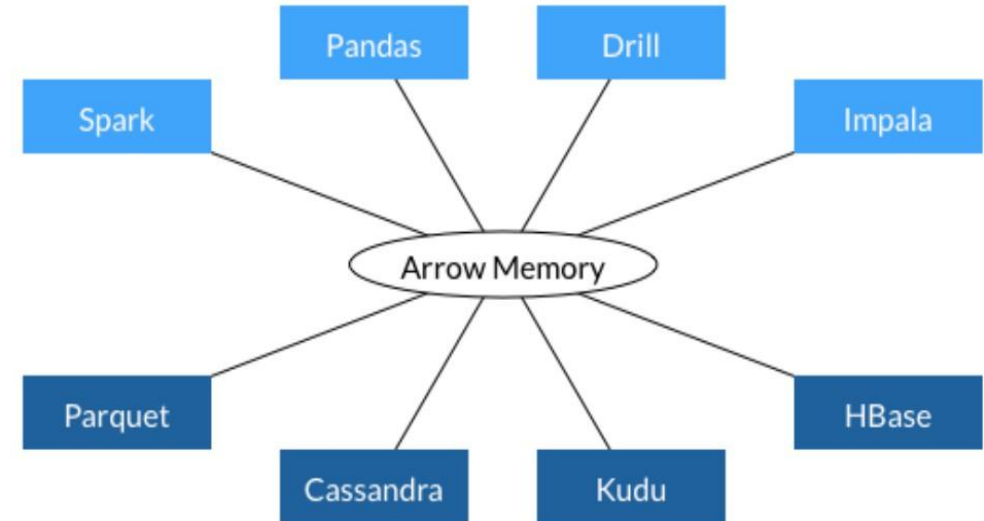
MLflow Models



Advantages of a Common Data Layer



- Each system has its own internal memory format
- 70-80% computation wasted on serialization and deserialization
- Similar functionality implemented in multiple projects



- All systems utilize the same memory format
- No overhead for cross-system communication
- Projects can share functionality (eg, Parquet-to-Arrow reader)

Example MLflow Model

```
mlflow.tensorflow.log_model(...)
```

my_model/
└── MLmodel

```
run_id:  
769915006efd4c4bbd662461  
time_created: 2018-06-28T12:34  
flavors:  
  tensorflow:  
    saved_model_dir:  
  estimator  
  signature_def_key:  
  predict
```

} Usable by tools that
understand TensorFlow model
format

} Usable by any tool that can
run Python (Docker, Spark,
etc!)

└── saved_model.pb
 └── variables/

Model Keras Flavor Example

K

```
mlflow.keras.log_model(  
...)
```

Train a model

mlflow

Model
Format

Flavor
1:



```
predict = mlflow.pyfunc.load_model(...)  
predict(pandas.input_dataframe)
```



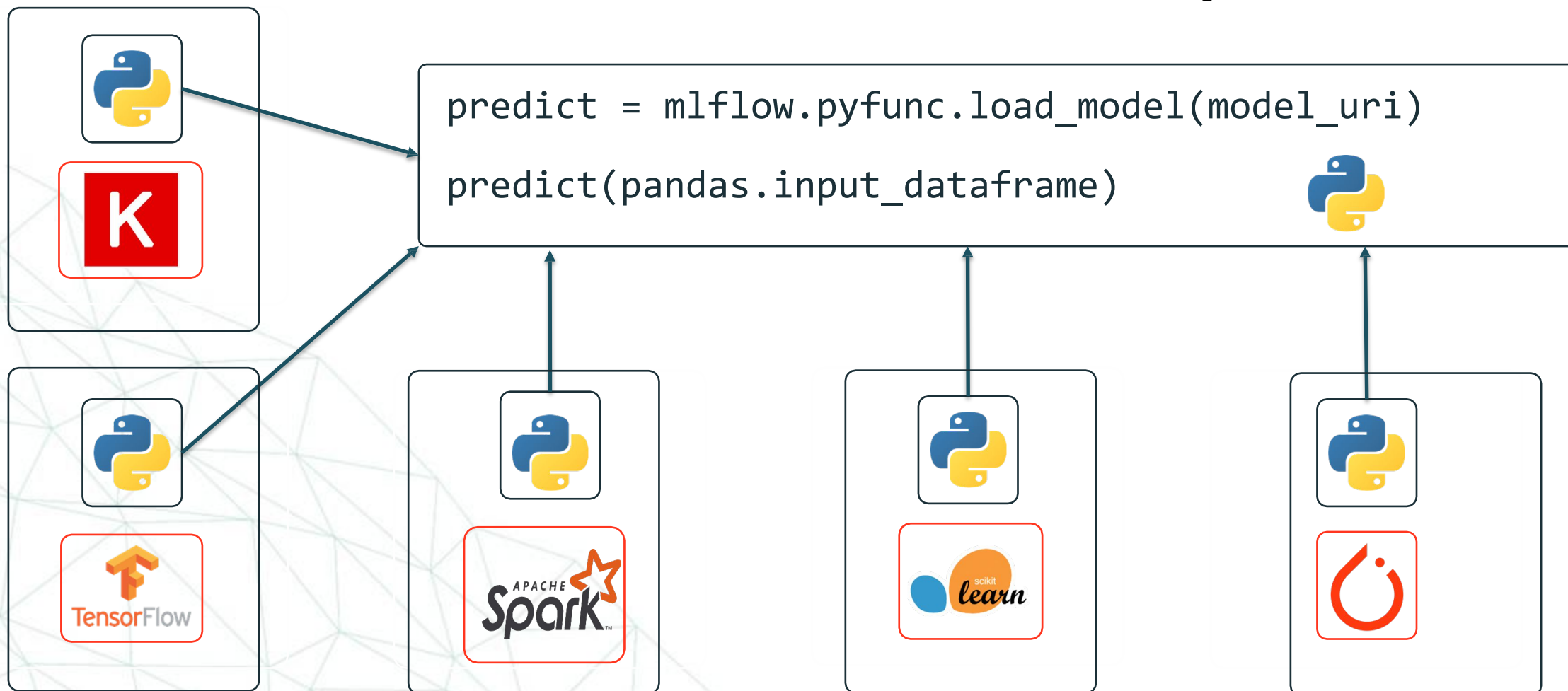
Flavor
2:



```
model = mlflow.keras.load_model(...)  
model.predict(keras.Input(...))
```



Model Flavors Example



MLflow Models

Packaging format for ML Models

- Any directory with MLmodel file

Defines dependencies for reproducibility

- Conda environment can be specified in MLmodel configuration

Model creation and loading utilities

- `mlflow.<model_flavor>.save_model(...)` or `log_model(...)`
- `mlflow.<model_flavor>.load_model(...)`

Deployment APIs

- CLI / Python / R / Java
- `mlflow models [OPTIONS] COMMAND [ARGS]...`
 - `mlflow models serve [OPTIONS [ARGS]`
....
 - `mlflow models predict [OPTIONS [ARGS]`
...

Learning More About MLflow

- `pip install mlflow` to get started
- Find docs & examples at mlflow.org
- Peruse code at [MLflow Github](https://github.com/mlflow/mlflow)
- Join the [Slack channel](#)
- More [MLflow tutorials](#)

Thank you!



mlflow

Platform for Complete Machine Learning Lifecycle

Part 3

Outline – Introduction to MLflow: Model Registry Workflows Explained – Part 3

- Concepts and Motivations
- MLFlow Component
 - MLflow Model Registry
 - Model Registry UI & API Workflow
 - Managed MLflow Model Registry Demo
 - Tutorials on local host
- Q & A

MLflow Components

mlflow Tracking

Record and query experiments: code, data, config, and results

databricks.com/mlflow

mlflow Projects

Package data science code in a format that enables reproducible runs on any platform



mlflow.org

mlflow Models

Deploy machine learning models in diverse serving environments



github.com/mlflow/mlflow

new

mlflow Model Registry

Store, annotate and manage models in a central repository



twitter.com/mlflow

The Model Management Problem

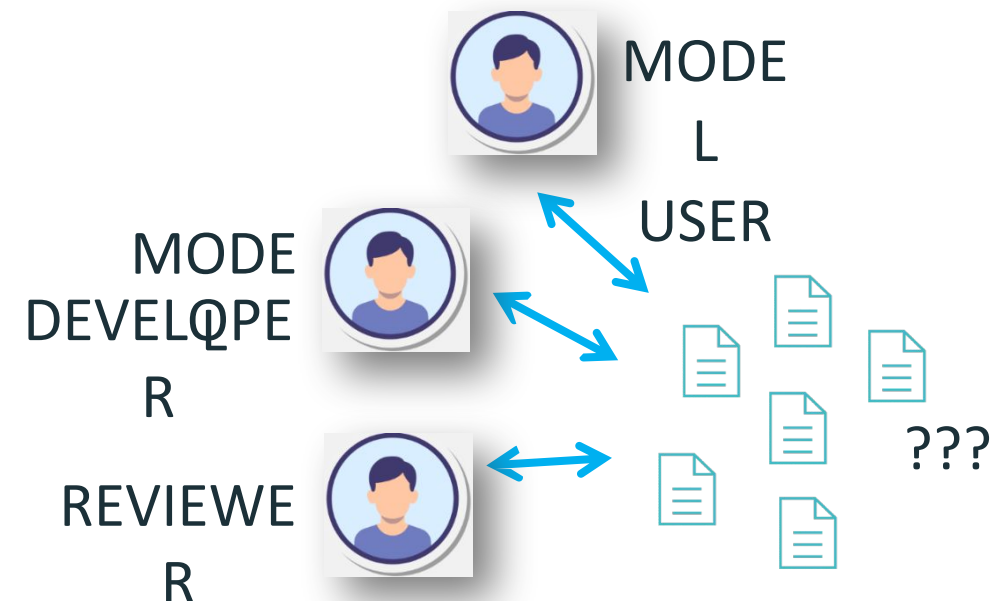
When you're working on one ML app alone, storing your models in files is manageable



The Model Management Problem

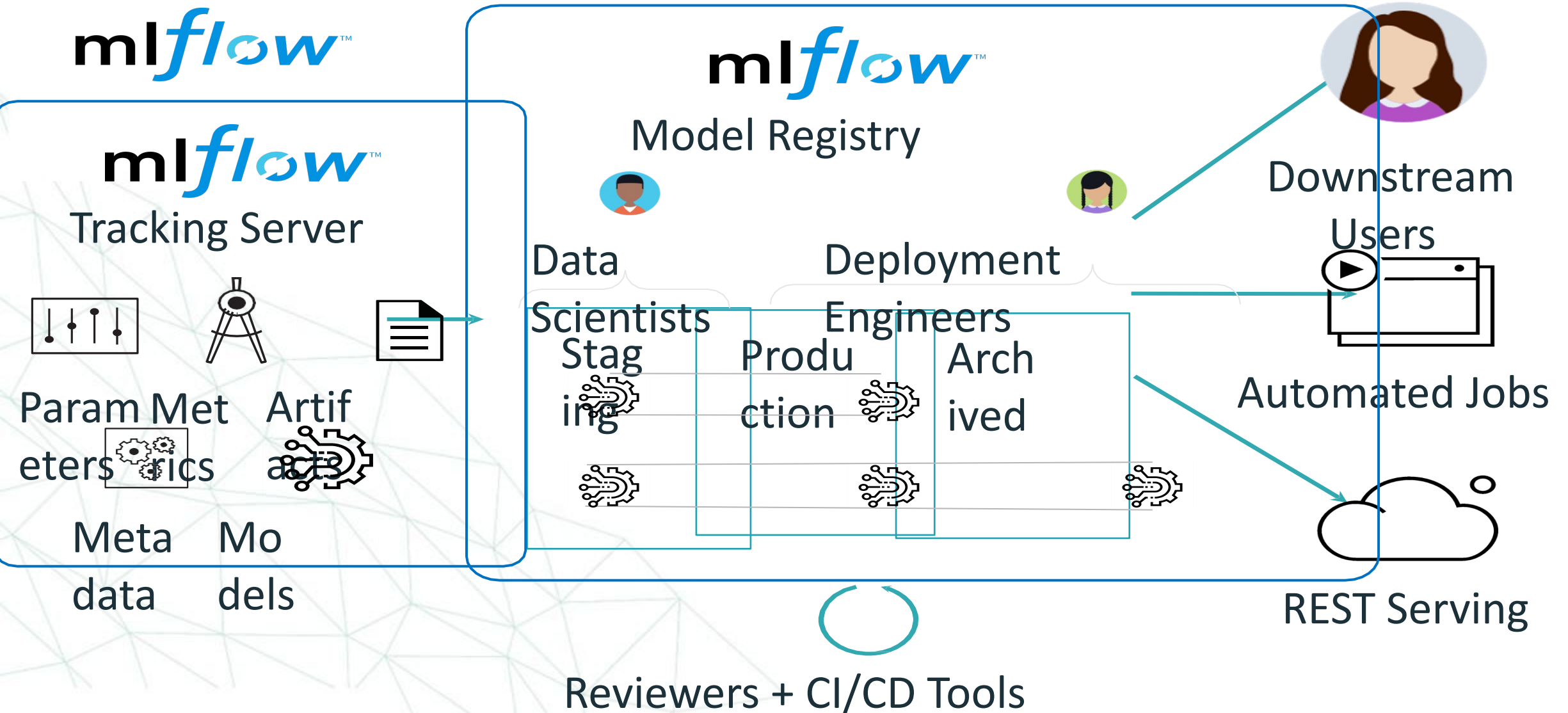
When you work in a large organization with many models, many data teams, management becomes a major challenge:

- Where can I find the best version of this model?
- How was this model trained?
- How can I track docs for each model?
- How can I review models?
- How can I integrate with CI/CD?



Model Registry

VISION: Centralized and collaborative model lifecycle management



MLflow Model Registry

- Repository of named, versioned models with comments & tags
- Track each model's stage: none, staging, production, or archived
- Easily inspect a specific version and its run info
- Easily load a specific version
- Provides model description, lineage and activities

Registered Models > Airline_Delay_SparkML ▾

Created Time : 2019-10-10 15:20:29

Last Modified : 2019-10-14 12:17:04

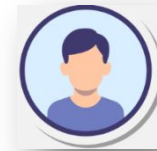
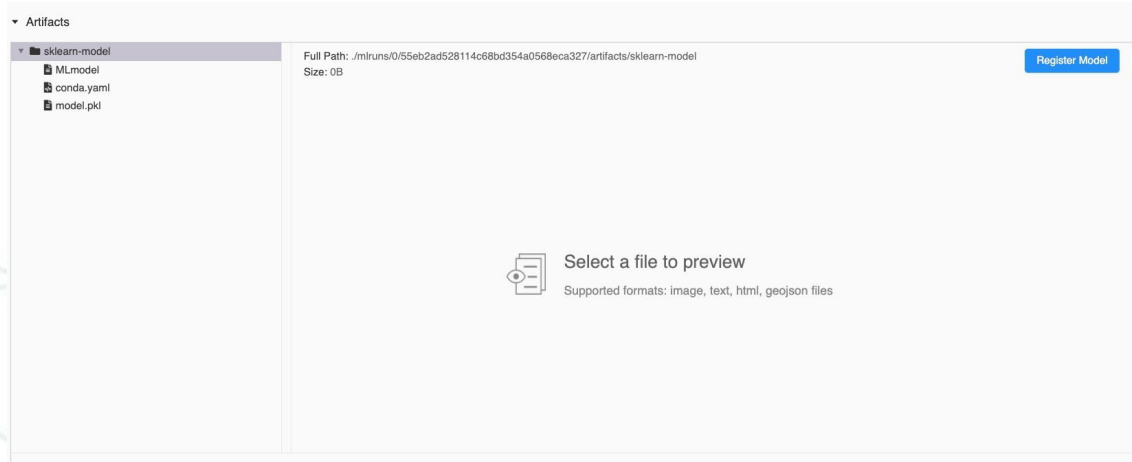
▾ Description

Predicts airline delays (in minutes) using the best Spark RF model from the AutoML Toolkit.

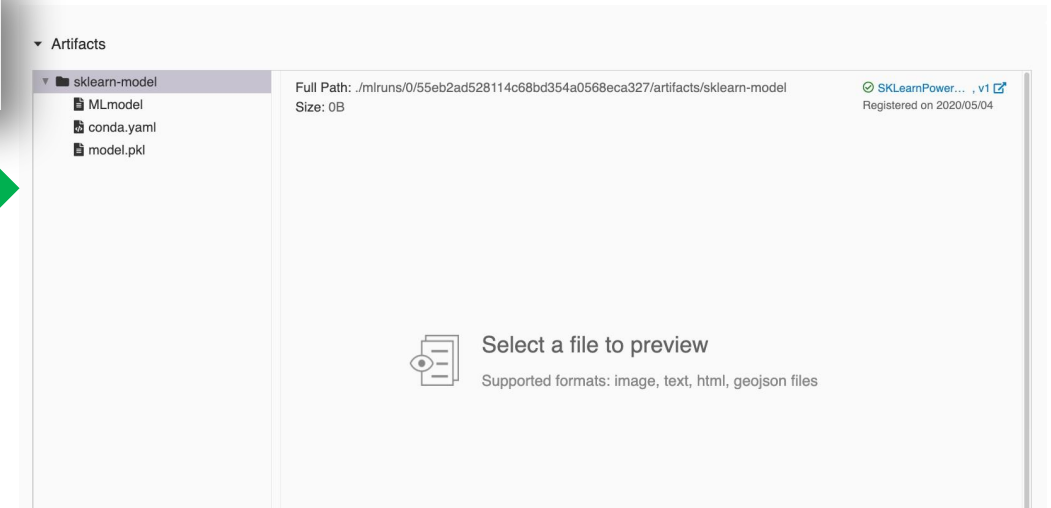
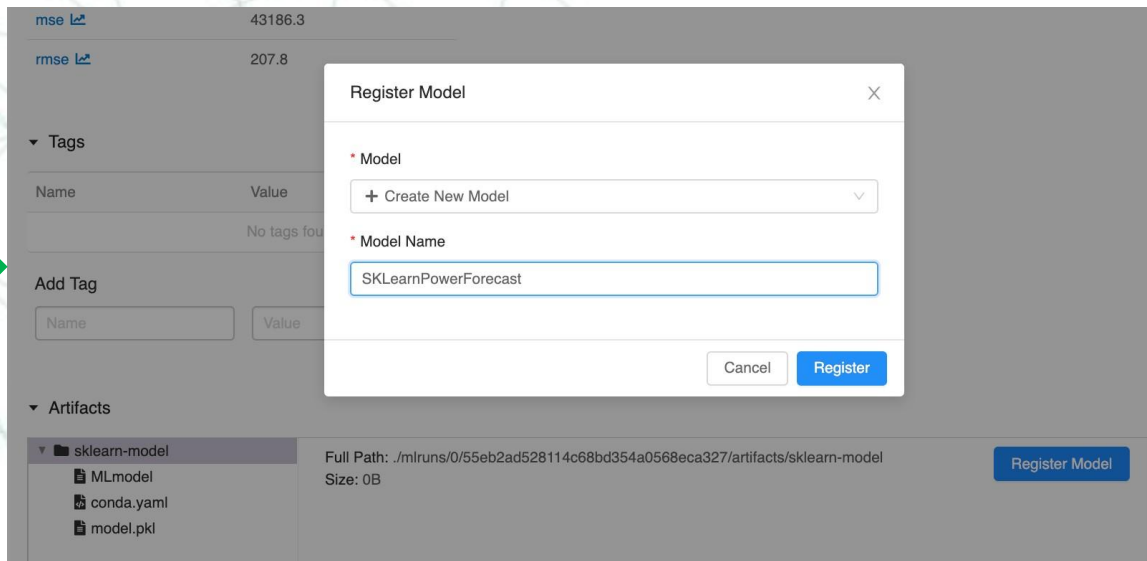
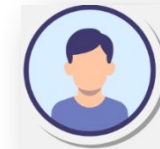
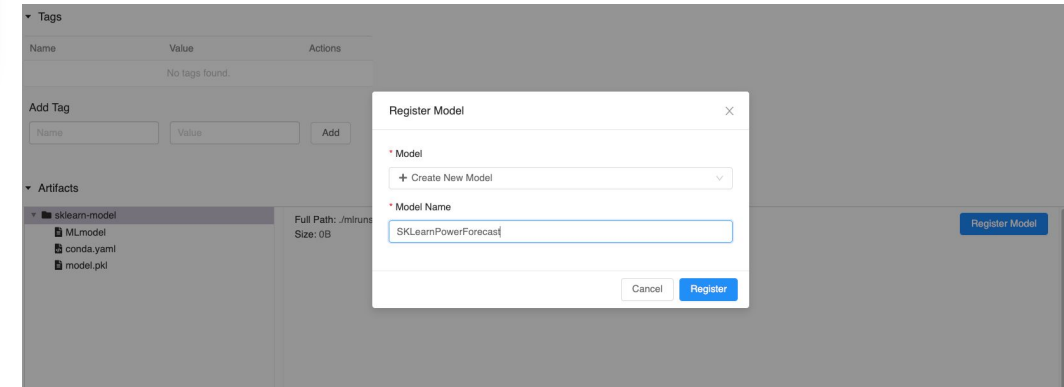
▾ Versions All Active(1)

Version	Registered at	Created by	Stage
 Version 1	2019-10-10 15:20:30	clemens@demo.com	Archived
 Version 2	2019-10-10 21:47:29	clemens@demo.com	Archived
 Version 3	2019-10-10 23:39:43	clemens@demo.com	Production
 Version 4	2019-10-11 09:55:29	clemens@demo.com	None
 Version 5	2019-10-11 12:44:44	matei@demo.com	Staging

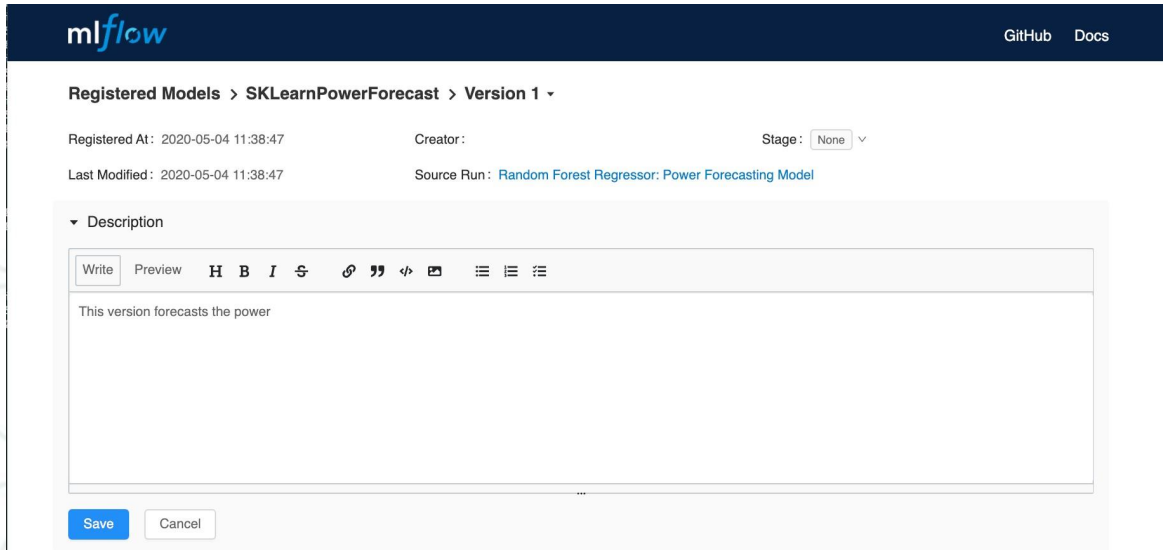
Model Registry Workflow UI



MODE
L
DEVELOPE
R



Model Registry Workflow UI



Registered Models > SKLearnPowerForecast > Version 1

Registered At: 2020-05-04 11:38:47 Creator: Stage: None

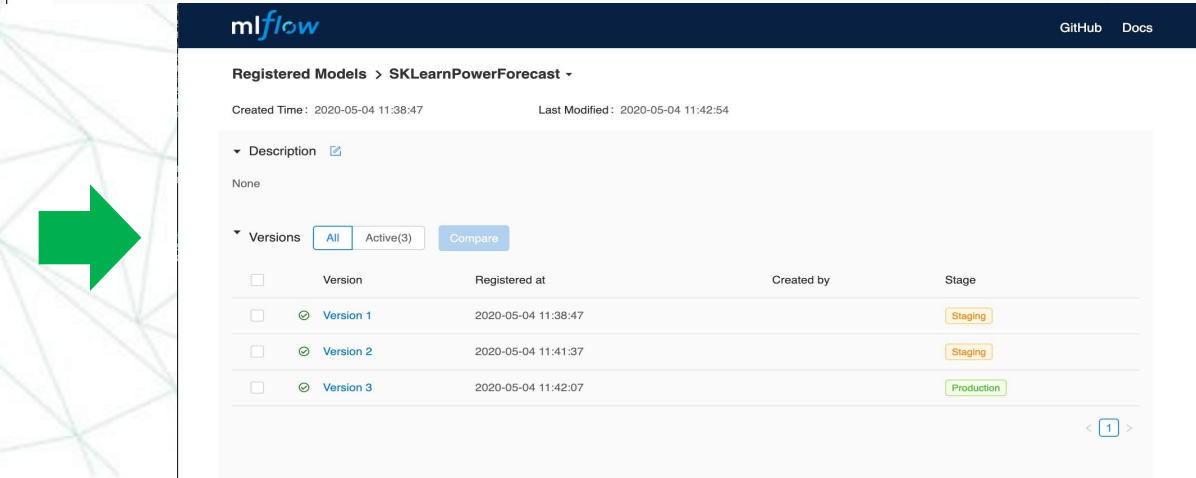
Last Modified: 2020-05-04 11:38:47 Source Run: [Random Forest Regressor: Power Forecasting Model](#)

▼ Description

Write Preview H B I S Link Quote Code Image List Bulleted List Numbered List

This version forecasts the power

Save Cancel



Registered Models > SKLearnPowerForecast

Created Time: 2020-05-04 11:38:47 Last Modified: 2020-05-04 11:42:54

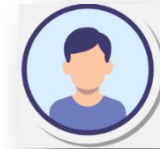
▼ Description [Link](#)

None

▼ Versions [All](#) [Active\(3\)](#) [Compare](#)

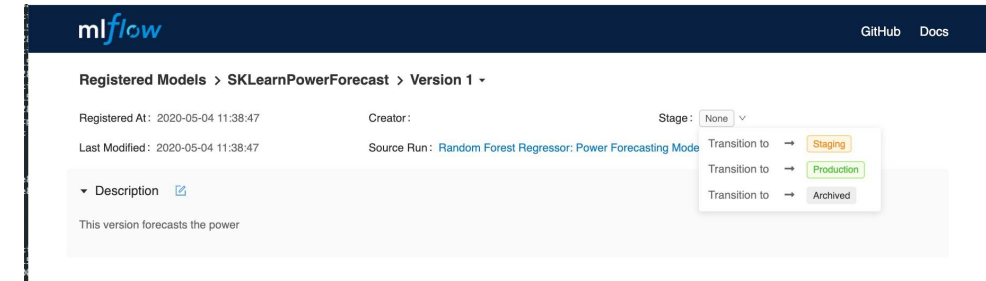
☐	Version	Registered at	Created by	Stage
☐	Version 1	2020-05-04 11:38:47		Staging
☐	Version 2	2020-05-04 11:41:37		Staging
☐	Version 3	2020-05-04 11:42:07		Production

< 1 >



MODE

L
REVIEWE
R



Registered Models > SKLearnPowerForecast > Version 1

Registered At: 2020-05-04 11:38:47 Creator: Stage: None

Last Modified: 2020-05-04 11:38:47 Source Run: [Random Forest Regressor: Power Forecasting Model](#)

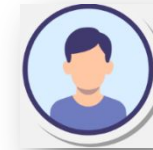
▼ Description [Link](#)

This version forecasts the power

Transition to → [Staging](#)

Transition to → [Production](#)

Transition to → [Archived](#)



DOWNSTREA
M USERS
AUTOMATED



JOBS REST

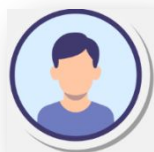


Model Registry Workflow API

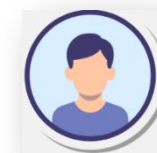
```
mlflow.register_model(model_uri, "WeatherForecastModel")
```

```
model_uri= "models://{model_name}/production".format(
    model_name="WeatherForecastModel")
model_prod = mlflow.sklearn.load_model(model_uri)
model_prod.predict(data)
```

MODEL
DEVELOPER



REVIEWERS
, CI/CD
TOOLS



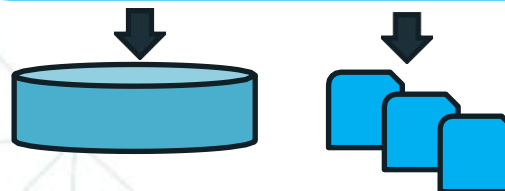
DOWNSTREAM
USERS



AUTOMATED
JOBS



REST
SERVING



```
client = mlflow.tracking.Mlflowclient()
client.transition_model_version_stage(name="WeatherForecastModel",
    version=5,
    stage="Production")
```

MLflow Backend Registry Stores

Entity (Metadata) Store and Models

- SQLStore (via SQLAlchemy)
 - PostgreSQL, MySQL, SQLite
 - Default is *mlruns.db* file locally
- Set programmatically for locally
- `mlflow.set_tracking_uri("sqlite:///mlruns.db")`
- `sqlite3 ./mlruns.db` (on local host)
- Managed MLflow on Databricks
 - MySQL on AWS and Azure

Artifact Store

- Local Filesystem
 - *mlruns* directory
- S3 backed store
- Azure Blob storage
- Google Cloud Storage
- DBFS artifact repo

```
(tutorials) → src git:(master) ✗ sqlite3 ./mlruns.db
```

```
SQLite version 3.30.1 2019-10-10 20:19:45
```

```
Enter ".help" for usage hints.
```

```
sqlite> .databases
```

```
main: /Users/julesdamji/gits/mlflow-workshop-part-3/src/./mlruns.db
```

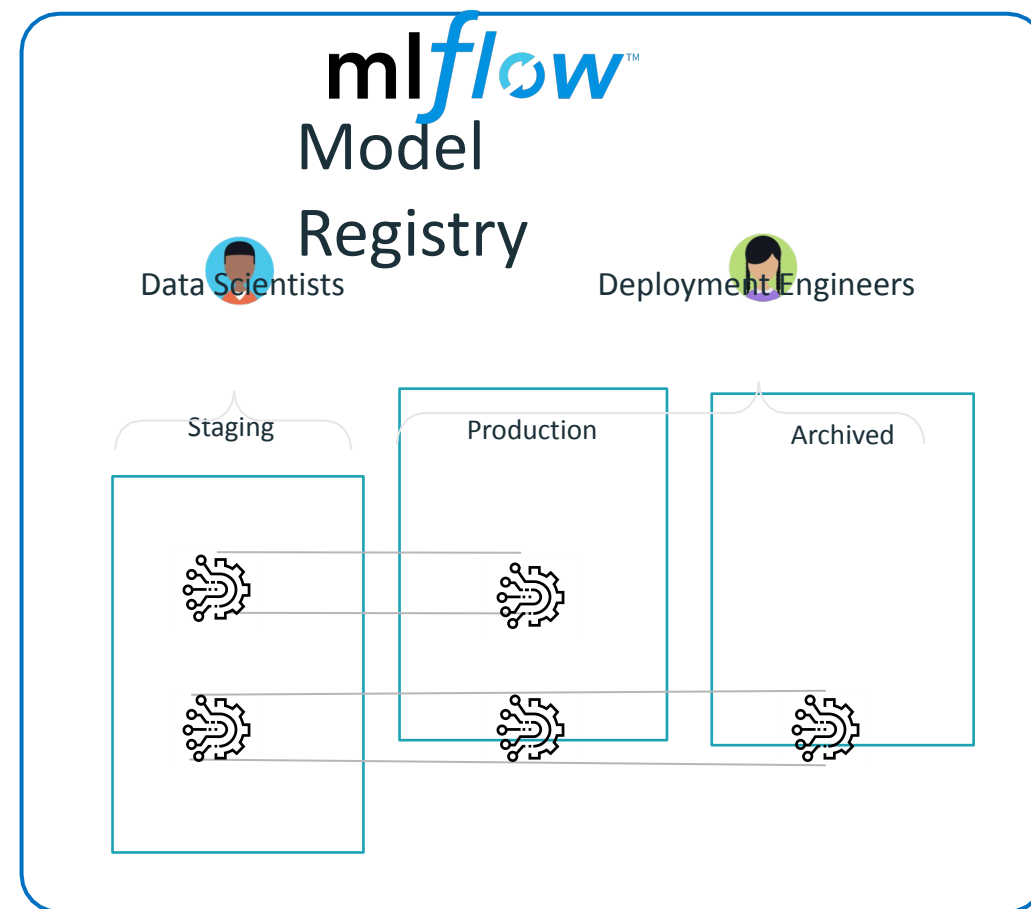
```
sqlite> .tables
```

alembic_version	latest_metrics	params	tags
experiment_tags	metrics	registered_models	
experiments	model_versions	runs	

```
sqlite> █
```

MLflow Model Registry Recap

- **Central Repository:** Unique named registered models for discovery across data teams
- **Model Registry Workflow:** Provides UI and API for registry operations
- **Model Versioning:** Allow multiple versions of model in different stages
- **Model Stages:** Allow stage transition: none, staging, production, or archived
- **CI/CD Integration:** Easily load a specific version for testing and inspection
- **Model Lineage:** Provides model description, lineage and activities



Recap of all parts: What Did We Talk About?

- Modular Components greatly simplify the ML lifecycle
- Easy to install & Great Developer experience
- Develop & Deploy locally; track locally or remotely
- Available APIs: Python, Java & R REST APIs and CLI tools
- Visualize experiments and compare runs
- Centrally register and manage model lifecycle

mlflowTM

Thank you!



Q & A